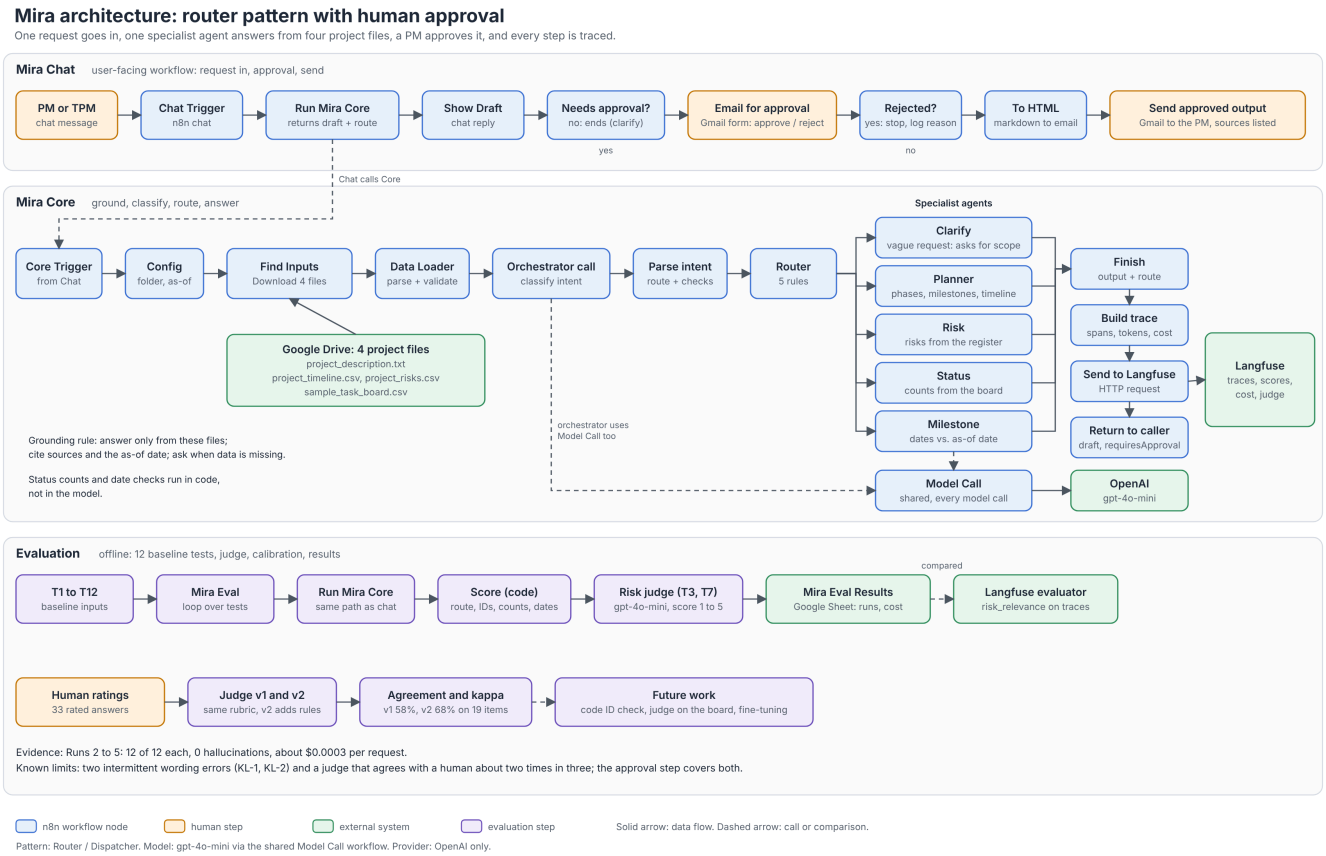
Mira Build Writeup

Design

Mira uses a router pattern: an orchestrator reads each request, a router sends it to one specialist agent, and a human approves the answer before it leaves the team.



Why a router. Requests differ (a plan, a risk view, a status report, a milestone question), and each needs different data and a different output format. A sequential pipeline would run every agent on every request, which costs more and gives the model room to invent content. A hierarchical design adds a planner layer that four request types do not need.

Agents.

- **Orchestrator:** classifies the request and returns an intent and a route.
- **Planner:** phases, milestones and timeline from the project description and timeline file.
- **Risk assessor:** a categorised risk view with impact and mitigation, drawn from the risk register.

- **Status reporter:** tasks done, in progress, to do and blocked, counted from the task board.
- **Milestone tracker (extended capability):** upcoming milestones and at-risk items against an as-of date.
- **Clarify:** replies to vague requests ('We want a chatbot') by asking for scope, goals and team size. It makes no model call and invents nothing.

Grounding and control. Every answer uses only the four project files and states its sources and as-of date. Counts and date comparisons are computed in code, not by the model. Plans, risk views and reports go to a PM for approval by email before they are sent.

Tool	Used for	Why this one
n8n Cloud	Workflows, routing, approval step	Low-code, with a chat trigger, sub-workflows and a Gmail approval wait built in
OpenAI gpt-4o-mini	Classification and generation	Low cost per request and approved on the project allow-list
Google Drive and Sheets	Project files in, evaluation results out	Where the files already live; easy to share the results
Langfuse	Traces, token cost, scores, judge evaluator	Shows each step and its cost, and runs an LLM-as-judge on traces
Gmail	Approval request and approved send	A reviewer decision with a reason and a timestamp

Implementation

Mira is six n8n workflows, and all 12 baseline tests passed in each of the four documented runs (Runs 2 to 5).

- **Mira Chat:** the user-facing chat, approval email and approved send.
- **Mira Core:** loads the four files from Drive, classifies the request, routes it and returns the answer with a trace.
- **Model Call:** one shared sub-workflow that calls gpt-4o-mini and returns the text, token counts and latency.
- **Mira Eval:** runs the 12 baseline inputs through Mira Core and scores them.

- **Mira Judge Calibration:** compares the risk judge with human ratings.
- **Mira Weekly:** runs the status report every Monday at 9:00 AM Pacific through Mira Core and the same approval email. Its first scheduled run fired on 2026-10-26 (n8n execution #750) and the approved report was delivered.

Core capabilities. The planner returns the eight documented phases with milestones and timeline, and does not invent goals. The risk assessor returns categorised risks with impact and mitigation from the 10-risk register. The status reporter counts tasks by status from the 25-task board and names the blocked task, T024. Vague inputs (T2, T4, T6, T9) get a request for scope, timeline, team size or task data, and no invented plan, risks or progress.

Extended capability. A milestone tracker answers T8 and T11 (blocked or at-risk tasks, milestones in the next two weeks) against an as-of date. The Sprint 2 update (T12) is answered by the status route and limited to Sprint 2 tasks; a separate stakeholder-update generator was not built. Human-in-the-loop is built in: a PM approves or rejects each plan, risk view or report by email, with a reason, before it is sent.

Observability. Every request builds a trace (the orchestrator call, the specialist call, tokens, latency, cost) and sends it to Langfuse. A Langfuse evaluator, `risk_relevance`, scores the risk answers as an LLM judge.

Baseline testing. The 12 inputs from the brief were run in Runs 2 to 5, and each run followed a change found in the one before.

Run	Change	Result
2	Orchestrator fix	12 of 12; T11 passed by mistake with an extra 2026-05-05 milestone in the window
3	Stricter T11 check and a prompt rule	12 of 12; T11 correct
4	Risk-relevance judge added for T3 and T7	12 of 12; judge scored both 5 of 5
5	Judge rule 8 (risks must be in the register)	12 of 12; an invented risk scored 3 and 4 before the rule and 2 and 2 after

Cost and evaluation

One project plan costs about $0.0006 and takes about 6 seconds, and the average request across all 12 tests costs $0.00032.

Cost per plan. In Run 5 the planner call used 918 prompt and 610 completion tokens, which is $0.0005 at gpt-4o-mini prices ($0.15 per million input tokens, $0.60 per million output tokens). The orchestrator call adds about $0.00009. A plan therefore costs about $0.00059, or about $0.06 per 100 plans. Risk answers cost about the same ($0.00058); status and milestone answers cost $0.0002 to $0.0003; a clarify reply costs only the orchestrator call.

Evaluation strategy. All 12 tests are scored by code (route, required items, task and risk IDs, counts, dates, and forbidden content). The two risk-relevance tests, T3 and T7, also go to an LLM judge, because relevance cannot be checked by rule. The judge was calibrated against human ratings on a baseline set and a separate test set.

Metric	Result	Source
Baseline tests passed	12 of 12 in each of Runs 2 to 5	Code scoring
Hallucinations (invented tasks, risks, milestones)	0 in Runs 2 to 5	Code scoring
Average latency	2.8 to 3.5 seconds per request	Run results
Cost per request	$0.000315 to $0.00032	Tokens and Langfuse
Risk judge on T3 and T7	5 of 5 in Runs 4 and 5	Judge, Langfuse evaluator
Judge agreement with a human	64% on 14 items; 58% (v1) and 68% (v2) on 19 test items	Calibration

Observations.

- Passing the baseline did not mean the answers were right in every detail. Two intermittent errors got through the code checks: a wrong project start date in the plan, and 'no blocked items' in a Sprint 2 update while T024 is blocked. The approval step is the control for both.

- The judge helps but is not reliable enough to replace a person. It agrees with a human about two times in three, and its task-ID rule in v2 wrongly rejected valid tasks. Checking IDs in code is the proposed fix.
- A test that passes can still be wrong: the first T11 run passed with an extra milestone in the window, so the check was tightened.
- Next steps: code checks for dates and IDs, a judge that sees the full task board, a second-model comparison once the allow-list allows it, and fine-tuning on corrected examples, which was too costly for this project.